

AES-GCM project Simple Hinglish guide

Angular + .NET Gateway + .NET API | Learning, maintenance aur future changes

Project owner: Faiz Noor | **Date:** 04 October 2026

Code baseline: d3a2b38 | **Branch:** codex/initial-setup

Humne kya banaya?

Login/register wali Angular app banayi hai. Request ka JSON Angular mein encrypt hota hai. Gateway encrypted envelope aage bhejta hai. API data decrypt karke kaam karti hai aur response encrypt karti hai. Angular response decrypt karke screen par dikhata hai.



Teen modules complete hain

Module	Kya ready hai
1. Foundation	Angular app, gateway/API projects, health check aur local routing.
2. Encryption	Session setup, AES-256-GCM request/response, encrypted echo, tamper/replay checks.
3. Authentication	Register, login, password hashing, protected profile aur token-revoking logout.

Is guide ko kaise padho

Pages 2-3 se flow samjho. Page 4 ko file map ki tarah use karo. Page 5 future feature add karne ka recipe hai. Pages 6-7 run commands, tests aur common errors ke liye hain.

Scope: Ye single-process learning demo hai. Users memory mein hain. API restart se accounts reset hote hain; browser refresh se login clear hota hai. Deployment mein authenticated HTTPS chahiye.

Encryption kaise chalti hai?

Pehle session banti hai, phir har message encrypt/decrypt hota hai.

Step 1: keys establish karna

- 1. Angular temporary P-256 public/private key pair banata hai. Public key **POST /api/crypto/session** ko bhejta hai. Private key browser mein rehti hai.
- 2. API apna temporary pair banati hai. Dono public keys se ECDH shared secret derive hota hai. API public key, random salt, session ID aur expiry return karti hai.
- 3. Dono sides HKDF-SHA256 se do alag AES-256 keys derive karti hain: request ke liye ek, response ke liye ek. AES keys network par send nahi hoti. Crypto session 10 minutes valid hai.

Step 2: encrypted round trip

- 4. Angular JSON ko bytes mein badalta hai, fresh 12-byte nonce banata hai, phir AES-GCM encrypt karta hai. Result ciphertext aur 16-byte tag hai.
- 5. Gateway envelope forward karta hai. API tag verify karke decrypt karti hai. Verification fail hui toh business handler run nahi hota.
- 6. API business result ko response key aur fresh nonce se encrypt karti hai. Angular matching request ID check karta hai, tag verify karta hai, phir response decrypt karta hai.

Term	Simple matlab
Ciphertext	Original JSON ka encrypted form.
Nonce / IV	Har encryption ka fresh random 12-byte value. Same key ke saath reuse mat karo.
Tag	Authentication check: data ya bound metadata badli toh decrypt fail hota hai.
AAD	Visible metadata jo tag se protect hoti hai: session ID, request ID, POST, route aur direction.
Base64	Bytes ko JSON mein carry karne ka encoding format. Ye encryption nahi hai.

Wire par kya dikhega?

```
{ "sessionId": "...", "requestId": "...",  
  "nonce": "Base64...", "ciphertext": "Base64...",  
  "tag": "Base64..." }
```

Session/request IDs aur route visible hain. JSON body encrypted hai. Session bootstrap aur health plaintext hain; isliye HTTPS bhi zaroori hai. Web Crypto tag ciphertext ke end mein deta hai; client usko split karta hai, kyunki .NET AesGcm separate tag leta hai.

Register se logout tak

Crypto session data protect karti hai. Login token user ki identity verify karta hai.

Register

Form name/email/password leta hai. AuthService encrypted transport call karta hai. API decrypt karke validation karti hai; email lowercase normalize hota hai. PasswordHasher salted password hash banata hai. Plain password store nahi hota. Registration ke baad user ko login karna hota hai.

Login

API password hash verify karti hai. Success par random 256-bit opaque token banta hai. Ye JWT nahi hai. Token encrypted response mein aata hai; Angular service memory mein rakhti hai. API token ka SHA-256 lookup hash store karti hai. Login 30 minutes valid hai.

Profile aur logout

Profile request ke encrypted JSON mein token jaata hai. API independently token validate karke sirf id/name/email return karti hai. Logout token record remove karta hai; us token se future profile request reject hoti hai. Route guard sirf navigation help hai; actual protection API mein hai.

POST route	Decrypted request	Application result
/api/auth/register	name, email, password	ok, message, user
/api/auth/login	email, password	ok, message, user, token
/api/auth/me	token	ok, message, user
/api/auth/logout	token	ok, message

Public routes: GET /api/health; POST /api/crypto/session. **Demo route:** POST /api/secure/echo encrypted payload receive karke echo karta hai.

Response samajhne ka rule

Valid encrypted request ka business result HTTP 200 ke andar encrypted aata hai. Wrong password bhi decrypted **ok: false** deta hai. Sirf HTTP 200 dekhkar login successful mat samjho. Invalid encrypted envelope jaise protocol errors plaintext HTTP 400/409 ho sakte hain.

Abhi data kahan hai?

API: users, salted hashes, token hashes aur crypto sessions memory mein. Angular: login token aur AES keys memory mein. localStorage, sessionStorage aur cookies use nahi hote. Auth credentials/tokens lab ke debug panels mein nahi dikhte.

Code ka map

Saare paths repository root se relative hain. Change se pehle related files saath padho.

File / location	Responsibility aur useful entry point
src/client/src/app/encrypted-transport.ts	connect/createSession: ECDH + HKDF. encrypt/decrypt: AES-GCM, tag split/join. post: complete encrypted HTTP round trip.
src/Demo.Api/Transport/SessionStore.cs	Create: session keys/expiry. Process: decrypt, replay check, handler run, encrypt result. Echo: Process ka demo wrapper.
src/Demo.Api/Auth/AuthStore.cs	Handle: auth operation dispatch. Register/LoginUser: validation/hash verification. Token lookup, expiry, logout, logout.
src/Demo.Api/Program.cs	Services register karta hai; health, crypto, echo aur auth routes map karta hai.
src/Demo.Gateway/Program.cs	Allowed routes check; request/response forward; timeout aur unavailable backend handle karta hai.
src/client/src/app/auth.service.ts	register/login/profile/logout calls; memory-only token aur user signal.
src/client/src/app/auth-page.ts + auth-page.html + auth-page.css	Login/register form, confirmation, submission status aur styling.
src/client/src/app/profile.ts	Protected profile verify karna aur logout.
src/client/src/app/app.routes.ts	login/register/profile/lab routes; profile navigation guard.
src/client/src/app/app.ts + app.html + app.css	App shell aur navigation.
src/client/src/app/lab.ts + lab.html + lab.css	Encrypted echo screen aur synthetic wire-payload debug panels.
src/client/proxy.conf.json	Development mein /api/** ko gateway:5100 par bhejta hai.
scripts/verify-transport.mjs + verify-auth.mjs	Real Web Crypto/.NET integration, tamper/replay aur authentication checks.

Reading order: AuthPage.submit() -> AuthService.login() -> EncryptedTransport.post() -> gateway -> API route -> SessionStore.Process() -> AuthStore.Handle() -> encrypted response.

Future change kahan karna hai?

Example: baad mein encrypted Update Profile feature add karna ho.

Naya encrypted endpoint add karne ka recipe

- 1. Contract:** Route decide karo, jaise POST /api/profile/update. Decrypted request mein token aur new name ho; response mein ok/message/user ho. Pehle exact field names decide karo.
- 2. Backend logic:** Profile service mein token validate karo, name validate karo, phir data update karo. Encryption session ko login proof mat samjho.
- 3. API route:** Demo.Api/Program.cs mein route map karo. SessionStore.Process(envelope, exactRoute, handler) se decrypted JSON handler tak aur result encrypted response tak pahunchao.
- 4. Gateway:** Demo.Gateway/Program.cs ki allowed-route list mein same POST path add karo. Ye step miss hua toh gateway 404 dega.
- 5. Angular:** AuthService mein transport.post(exactRoute, payload) use karo. Form/component aur zarurat ho toh app.routes.ts mein page add karo. Token ko auth service ke andar rakho.
- 6. Verification:** Build aur integration checks run karo. Valid update, invalid token, validation failure, tampering aur replay cover karo. Phir module ka Git commit/push karo.

Common changes ka shortcut

Change	Files / attention
Colors, form layout, labels	auth-page.html/css; app.html/css; lab.html/css.
New register field	Form + AuthService payload + Credentials/validation + public response + tests.
API port change	API Properties/launchSettings.json + gateway ApiBaseUrl setting.
Gateway port change	Gateway launchSettings.json + client proxy.conf.json.
Crypto session expiry	SessionStore.Create: AddMinutes(10). Client renewal/expiry behavior recheck karo.
Login expiry / password policy	AuthStore: AddMinutes(30), validation; Angular form/help text aur tests update karo.
Database add karna	AuthStore users dictionary ko persistent repository se replace karo. Password hashes hi save karo; token expiry/revocation preserve karo.

Crypto rule: AAD ka method/path/direction, HKDF labels, key length ya envelope format change karo toh Angular aur .NET dono update karo. Known interoperability/tamper tests dobara run karo.

Project run aur test karna

Windows PowerShell mein repository root se start karo. Teen alag terminals chahiye.

Setup aur startup

Installed project baseline: Angular 21; Node 20.19+ compatible baseline; .NET 8 target. .NET SDK ke saath .NET 8 runtime bhi available hona chahiye. Client dependencies ek baar install karo:

```
cd src/client
npm ci
cd ../../
```

Terminal 1 - API:

```
dotnet run --project src/Demo.Api
```

Terminal 2 - gateway:

```
dotnet run --project src/Demo.Gateway
```

Terminal 3 - Angular:

```
cd src/client
npm start
```

Browser: **<http://localhost:4200/register>** -> account banao -> /login -> /profile -> logout. /lab par sample message encrypt karke wire aur decrypted result dekho.

Build aur automated checks

```
# Repository root
dotnet build AesGcmDemo.sln
# Client directory
cd src/client
npm run build
cd ../../
# API + gateway running hone chahiye
node scripts/verify-transport.mjs
node scripts/verify-auth.mjs
```

Angular dev proxy bhi exercise karna ho toh Angular running rakho aur root se:

```
$env:DEMO_URL = 'http://localhost:4200'
node scripts/verify-auth.mjs
Remove-Item Env:DEMO_URL
```

Verified baseline: .NET/Angular builds aur transport/auth integration checks pass hue. Scripts Node Web Crypto se actual client transport run karte hain. Browser clicks ki automated verification host browser-control runtime failure ki wajah se pending thi.

Errors, limits aur reference

Problem aaye toh pehle ye checks dekho; phir related file map use karo.

Symptom	Likely reason / action
Connection failed / HTTP 502	API:5200 aur gateway:5100 running hain? ApiBaseUrl sahi hai?
HTTP 504	Backend gateway ke 10-second timeout ke andar respond nahi kar raha. API logs dekho.
HTTP 404 on new route	Gateway allowlist aur API route exact match karo.
HTTP 400 / decrypt failed	Nonce/tag size, ciphertext, AES key, AAD route/direction aur HKDF labels match karo. Envelope edit hua ho toh reject expected hai.
HTTP 409	Unknown/expired crypto session, replayed request ID ya request capacity. API restart hua ho toh browser refresh karke login dobara karo.
Login refresh ke baad gaya	Expected: browser token memory-only hai. Dobara login karo.
Registered account missing	API restart ne in-memory users clear kiye. Dobara register karo.
ok:false / too many attempts	Register/login ka global 60 attempts/minute budget. Minute wait karo. Five wrong passwords se account one minute locked hota hai.
Build file locked	Running API/gateway apne terminals mein Ctrl+C se stop karo; build, phir restart karo.

Current demo limits

Crypto session: 10 min; login: 30 min; 1,000 active crypto sessions; 1,000 accepted requests/session; 1,000 users aur login records each. Bootstrap global 30 requests/minute. Request bodies 32 KiB max; client plaintext 16,000 bytes max. Sensitive responses no-store hain.

Deployment se pehle

Authenticated HTTPS, supported patched runtimes, persistent identity storage aur multiple API instances ke liye shared-state design chahiye. Browser mein malicious JavaScript chale toh AES usse protect nahi karega. Current operations simple locks se serialize hoti hain; ye high-traffic architecture nahi hai.

Project aur references

Repository:

github.com/noorfaiz115/angular-dotnet-aes-gcm-demo

Project docs: README.md; docs/architecture.md; docs/module-2.md; docs/module-3.md. Ye guide current source code ko explain karti hai; suggested Update Profile feature abhi implement nahi hai.

API reference: [Microsoft AesGcm](#) | [Microsoft PasswordHasher](#) | [W3C Web Crypto](#)